Data Science & AI

DATA STRUCTURES Through Python

Hash Tables

Lecture No.- 01

By- Satya sir

Recap of Previous Lecture

Doubly Linked List

- Node structure
- Construction
- Insertion into DLL
  - At front
  - At End
  - At middle
- Deletion from DLL
  - Middle Node
  - First Node
  - Last Node

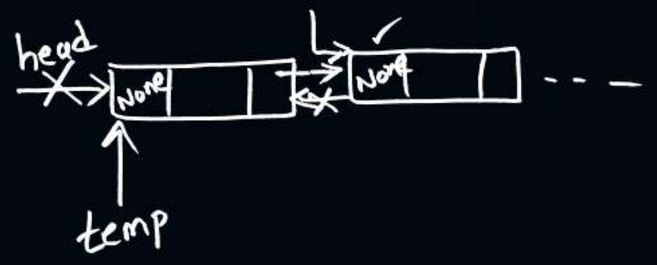
Topics to be Covered

- Deletion of first Node & last Node in DLL

- Hash Tables
  - Hashing ?
  - Hash Function
  - Collission ?
  - Collission Resolution Techniques
    - Open hashing / chaining method

Topic : DLL – Insertion, Deletion

Deletion of first Node in DLL/with or without Tail

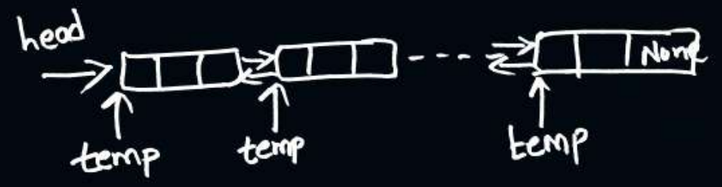



```
def Delete_first(self):
    if self.head is None:
        Print('Empty list')
        return
    elif self.head.next is None:
        temp = self.head
        self.head = None # self.Tail = None
        temp = None
    # End of elif
    else:
        temp = self.head
        temp.next.prev = None
        self.head = self.head.next
        temp = None
```

$\boxed{\text{Time Complexity} = O(1)}$

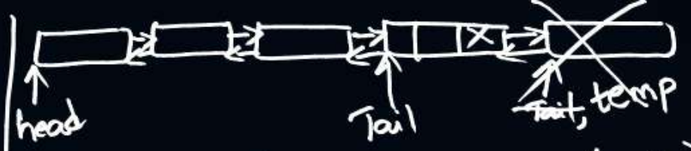
Topic : DLL – Insertion, Deletion

Deletion of Last Node without Tail




```
def delete_last(self):
    if self.head is None:
        Print('Empty list')
        return
    elif self.head.next is None:
        temp= self.head
        Self.head= None
        temp = None
    else:
        temp = Self.head
        while temp.next is not None:
            temp=temp.next
        temp.Prev.next = None
        temp= None
```

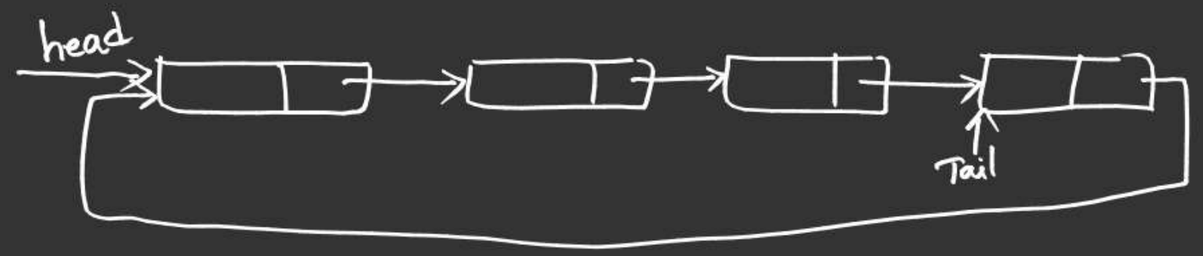
Time Complexity : $O(n)$




```
def delete_last_withTail(self):
    if self.head is None:
        Print('Empty list')
        return
    elif self.head.next is None:
        temp= Self.head
        Self.head, Self.Tail= None, None
        temp= None
    else:
        temp= Self.Tail
        Self.Tail= Self.Tail.Prev
        Self.Tail.next = None
        temp= None
```

Time Complexity == $O(1)$

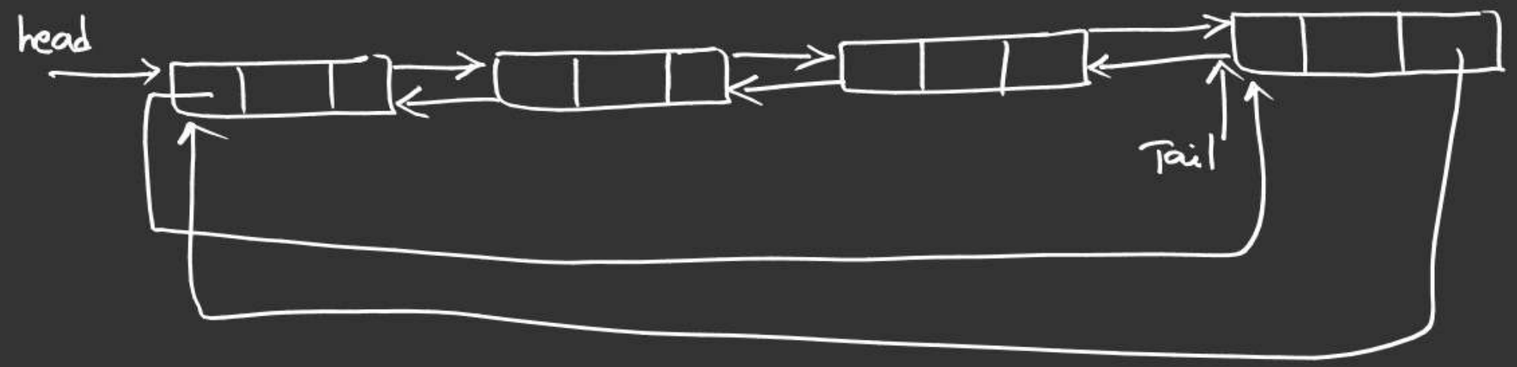
Circular SLL Structure



Last Node

temp.next == self.head

Circular DLL Structure



Topic : CLL – Insertion, Deletion

Circular Singly Linked List with Tail

SLL

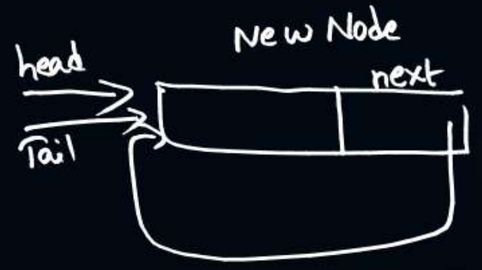
```
class Node:
  def __init__(self,data):
    self.data = data;
    self.next = None;
# Self.Prev = None    DLL

class CreateList:
  #Declaring head and tail pointer as null.
  def __init__(self):
    self.head = Node(None);
    self.tail = Node(None);
    self.head.next = self.tail;
    self.tail.next = self.head;
```

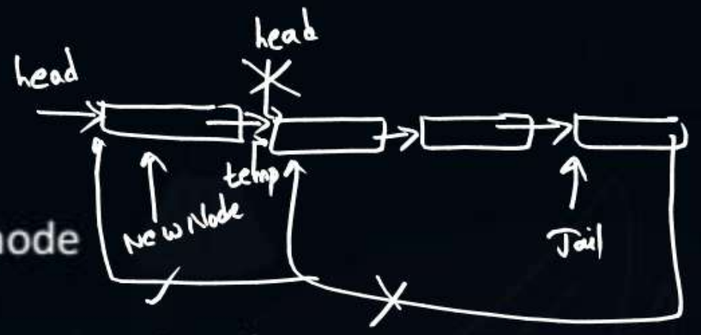
OR

Self.head = None

Self.Tail = None




```
def addAtStart(self,data):
  newNode = Node(data);
  #Checks if the list is empty.
  if self.head.data is None:
    #If list is empty, both head and tail would point to new node.
    self.head = newNode;
    self.tail = newNode;
    newNode.next = self.head;
  else:
    #Store data into temporary node
    temp = self.head;
    #New node will point to temp as next node
    newNode.next = temp;
    #New node will be the head node
    self.head = newNode;
    #Since, it is circular linked list tail will point to head.
    self.tail.next = self.head;
```



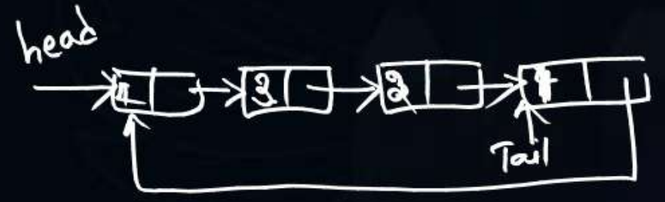
Topic : CLL – Insertion, Deletion

```
def display(self):
   current = self.head;
   if self.head is None:
     print("List is empty");
     return;
   else:
       #Prints each node by incrementing pointer.
      print(current.data),
      while(current.next != self.head):
         current = current.next;
         print(current.data),
      print("\n");
```

Driver Code

```
class CircularLinkedList:
  cl = CreateList();

  cl.addAtStart(1);
  cl.display();
  cl.addAtStart(2);
  cl.display();
  cl.addAtStart(3);
  cl.display();
   cl.addAtStart(4);
  cl.display();
```



Topic : Hash Tables - 1

		Insertion			Deletion		
		At Front	At middle	At Last	First Node	Middle Node	Last Node
SLL	Without Tail	O(1)	O(n)	O(n)	O(1)	O(n)	O(n)
	With Tail	O(1)	O(n)	O(1)	O(1)	O(n)	O(n)
DLL	Without Tail	O(1)	O(n)	O(n)	O(1)	O(n)	O(n)
	With Tail	O(1)	Exactly middle Position O(n)	O(1)	O(1)	Exactly middle Node O(n)	O(1)
CLL	Without Tail	O(1)	O(n)	O(n)	O(1)	O(n)	O(n)
	With Tail	O(1)	O(n)	O(1)	O(1)	O(n)	O(n)

Topic : Hash Tables - 1

Hashing

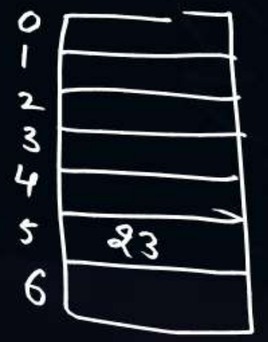
- The Process of mapping key to it's value
- Value is the memory location / Slot Number / bucket number / Block Number in a region of Memory, Hash Table.
- To map key to value, a Pre defined, mathematical function is Used; Called as Hash Function

Ex: Hash function : $h(k) = (k+3) \% n$ n = size of hash Table, k = key

$n = 7$ $h(23) = (23+3) \% 7$

$= 26 \% 7$

$= 5$



Topic : Hash Tables - 1

Collision Resolution Techniques

- The size of Hash Table will be limited/bounded.
- So, when more than 1 key maps to same value/slot, it is said to be collision.
- To Resolve collissions, Either of 2 Techniques can be followed.

1) Open Hashing / Seperate Chaining method

2) Closed Hashing / Open addressing
   - Linear Probing
   - Quadratic Probing
   - Double Hashing

Topic : Hash Tables - 1

Chaining method

- It uses SLL Implementation.
- All keys that maps to same value are organized as chain of elements using Linked List.
- One slot can accomodate any number of keys.
- As Linked Lists support sequential access, Access Time Increases.
- Demands Extra memory space in Each node/key for next Node reference.

Topic : Hash Tables - 1

Example :

Let Keys = {32, 45, 73, 37, 53, 17, 21, 60} Table Size = 7

$h(K) = K \% 7$

$h(32) = 32 \% 7 = 4$

$h(45) = 45 \% 7 = 3$
$h(73) = 73 \% 7 = 3$ } collision

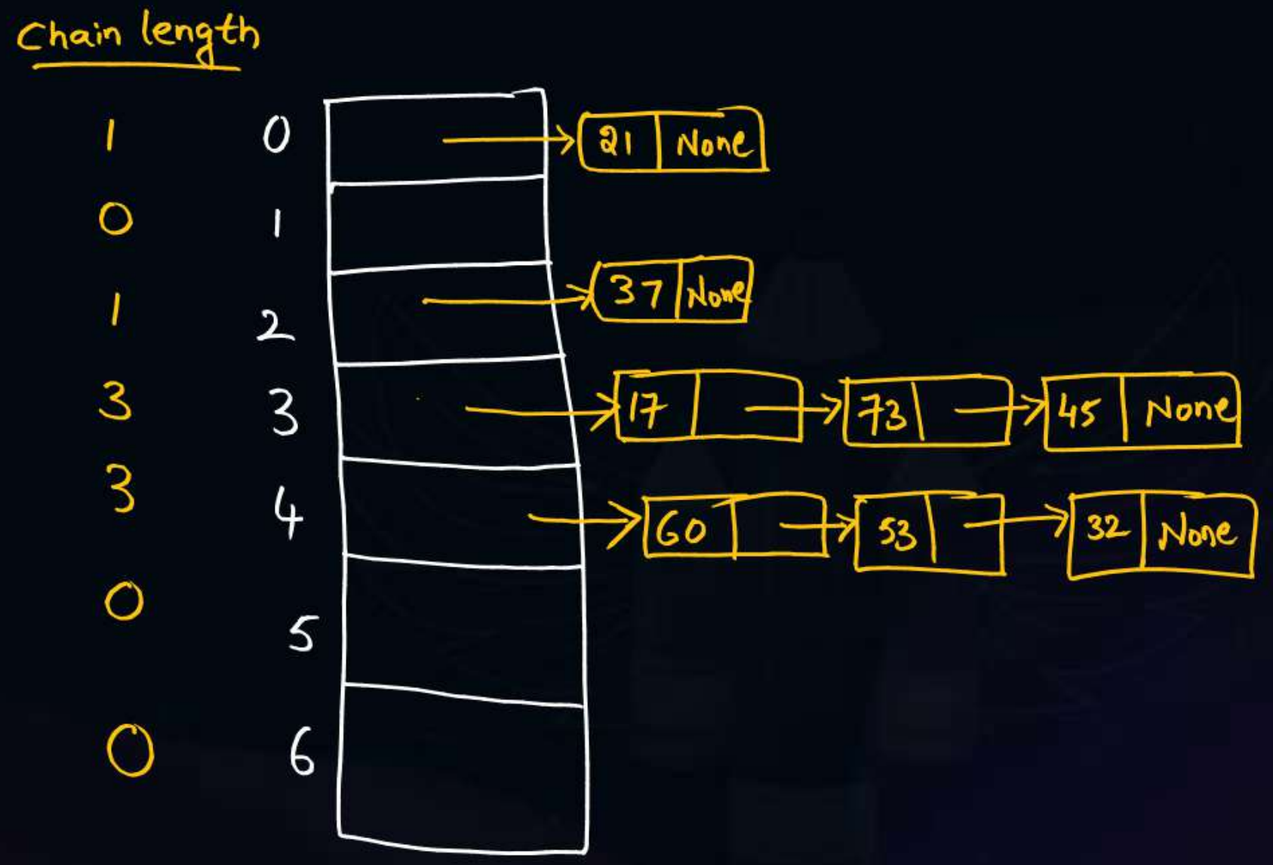
$h(37) = 37 \% 7 = 2$

$h(53) = 53 \% 7 = 4$

$h(17) = 17 \% 7 = 3$

$h(21) = 21 \% 7 = 0$

$h(60) = 60 \% 7 = 4$



2 mins Summary

- DLL Deletion
  - At End
  - At front
- CLL Introduction, Insertion At front
- Time Complexities
- Hashing, Hash Function, Hash Table
- Collision Resolution Techniques
  - Chaining Method.

To be contd ...

THANK - YOU